Rapport d'avancement : Étude de rupture de charge entre Cypher 25 et SQL sur le Knapsack Path Problem

JIN Christophe - PAN Alexis

1. Introduction

Nous souhaitions travailler sur les bases de données en cherchant un sujet techniquement stimulant. De ce fait, nous avons sollicité l'aide du professeure Mme Amélie Gheerbrant. Elle nous a d'abord orientés vers plusieurs papiers de recherche, puis nous a proposé d'analyser les rapports de projets des étudiants de M2 portant sur la comparaison entre **Cypher 25**, **Cypher 5** et **SQL**.

Après analyse de ces résultats, nous avons constaté que si Cypher 25 facilite grandement l'écriture de requêtes grâce à la nouvelle norme GQL, les tests de performance restaient limités à des cas d'usage classiques (comme les données de transport). Les limites réelles de la gestion mémoire, notamment face à des problèmes d'optimisation complexes, n'avaient pas été testées, de ce fait, nous avons décidé d'évaluer les limites de gestion mémoire entre le modèle relationnel (**SQL/PostgreSQL**) et le modèle de graphe (**Cypher 25/Neo4j**) face à un problème **NP-Difficile** : le *Problème du chemin du sac à dos* (Knapsack Path Problem).

Pourquoi cette étude ?
Habituellement, on compare SQL et Cypher sur des tâches simples (comme ce que les M2 ont fait dans leurs projets). Mais avec l'arrivée de **Cypher 25**, on peut désormais résoudre des problèmes beaucoup plus complexes (comme le sac à dos) directement dans la base de données.

Le but est de comparer deux types de stratégies opposées pour gérer une surcharge de données :

1. **La stratégie de Neo4j (Tout en mémoire) :** Neo4j essaie d'être "intelligent". Il analyse les chemins et abandonne immédiatement ceux qui dépassent le budget pour gagner du temps. Cependant, il effectue tous ses calculs en mémoire vive (RAM). Si le problème devient trop complexe, la mémoire sature et le système s'arrête brutalement (Erreur Out Of Memory).
2. **La stratégie de PostgreSQL (Débordement disque) :** PostgreSQL n'est pas un spécialiste des graphes, mais il est très robuste. S'il n'a plus assez de place en mémoire vive pour stocker ses calculs, il est capable d'utiliser le disque dur pour continuer à travailler. C'est beaucoup plus lent, mais cela lui permet potentiellement de terminer le calcul là où Neo4j échouerait.

En résumé, nous voulons savoir si, face à un graphe piégé complexe, rempli de mauvaises pistes. Le graphe propose des chemins qui semblent excellents au début (gros bénéfices), mais qui finissent par dépasser le budget. Ce "piège" force les deux bases de données à explorer des milliers de branches inutiles et à saturer leur mémoire. L'enjeu n'est donc plus de savoir quel logiciel est le plus rapide, mais lequel est le plus robuste.
Ce qui permet poser la question : *Est-ce que la gestion brutale sur disque de SQL finit par battre l'élagege intelligent (mais gourmand en RAM) de Cypher 25 sur un problème NP-Difficile ?*

Métrique : Comment mesurer le succès ?
Le projet sera réussi si nous parvenons à répondre à ces trois questions clés :

— **Le point de rupture de Neo4j :** À partir de quelle taille de graphe la mémoire vive (RAM) sature-t-elle complètement, provoquant l'arrêt du calcul sur Neo4j ?

— **La limite d'endurance de PostgreSQL :** Est-ce que PostgreSQL parvient à finir le calcul en utilisant le disque dur quand la mémoire est pleine, et à quel point cela ralentit-il l'exécution ?

— **L'impact du "piège" :** Quelle est la différence de temps et de mémoire consommée entre un graphe normal et notre graphe "labyrinthe" ? Cela nous permettra de mesurer à quel point nos fausses pistes ont réussi à tromper les optimiseurs.

2. Implémentation

2.1 État actuel du développement

Le coeur de notre travail repose sur la conception d'un **générateur de graphes "pièges"** codé en Python. Pour le moment, nous avons finalisé la modélisation théorique de notre protocole : nous avons défini la structure mathématique du graphe et la distribution des coûts pour garantir que les moteurs de recherche soient poussés dans leurs retranchements.

Le principe du graphe est le suivant : nous plaçons des récompenses (bénéfices) très attractives dès le début de certains chemins. Cette stratégie force l'ordinateur à explorer ces pistes en priorité, les considérant comme prometteuses. Cependant, ces chemins sont des impasses calculées qui finissent par coûter trop cher et dépasser le budget autorisé.

Ce mécanisme oblige les bases de données à mémoriser et traiter des milliers de fausses pistes avant de réaliser qu'elles sont mauvaises. Ce test nous permettra d'observer quel système sature en premier.

2.2 Structure

Le projet est organisé autour d'un flux de données automatisé permettant de passer de la création du piège à l'analyse des résultats :

- **Création du piège :** On fait un script qui utilise la bibliothèque `NetworkX` pour construire la structure du graphe. Il ne se contente pas de créer des points et des traits, il calcule précisément les valeurs de "coût" et de "profit" sur chaque arête pour s'assurer que les fausses pistes soient mathématiquement plus tentantes que le chemin optimal.
- **Les tests :** Pour analyser et tester les résusltats cela se fera en trois étapes :
  1. L'importation du graphe généré dans PostgreSQL et Neo4j.
  2. L'exécution des requêtes de recherche (SQL vs Cypher).
  3. La surveillance en temps réel de la consommation de mémoire vive et l'enregistrement des temps d'exécution.
- **La logique de calcul (Représentation des données) :**
  - **En SQL :** Nous utilisons une requête "boucle" (`WITH RECURSIVE`). À chaque étape, PostgreSQL crée une table temporaire pour stocker les chemins explorés. C'est cette table qui finira par déborder sur le disque dur si la mémoire sature.
  - **En Cypher 25 :** Nous utilisons les nouveaux modèles de chemins (*Quantified Path Patterns*). L'opérateur `allReduce` agit comme un sac à dos virtuel qui se remplit au fur et à mesure de la marche dans le graphe, calculant le score total en temps réel pour décider s'il faut continuer ou abandonner un chemin.

2.3 Précisions techniques

Choix des données (Entiers ou Flottants) : Pour nos premiers tests, nous utilisons des **nombres entiers** pour les coûts et les bénéfices. Cela nous permet de valider la logique de calcul sans erreurs d'arrondi. Cependant, nous prévoyons de passer aux **nombres à virgule flottante** (décimaux) par la suite. Ce changement augmentera naturellement la taille des données stockées en mémoire, accélérant ainsi le point de saturation des systèmes.

Stratégie d'arrêt (L'élagage intelligent) : Nous ne nous contentons pas de vérifier le budget à la fin du chemin. Nous intégrons la condition d'arrêt directement au cœur de la recherche Cypher (`WHERE total_cost < budget`).

Cela active l'*Early Pruning* (l'élagage précoce) : dès qu'un chemin dépasse le budget, Neo4j l'abandonne immédiatement. Cette approche place Neo4j dans sa configuration la plus "intelligente" et nous permet de tester si cette intelligence suffit à compenser la consommation de RAM.

Sémantique des chemins (Exploration sans limites) : Nous avons choisi la règle du `ANY PATH` (n'importe quel chemin). Contrairement à une recherche classique qui s'interdit de repasser par le même point, cette option autorise l'ordinateur à revisiter des nœuds tant qu'il reste du budget.

Cela multiplie de façon exponentielle le nombre de parcours possibles. C'est le réglage le plus sévère pour les deux moteurs : il transforme notre graphe en un labyrinthe quasi infini, poussant la RAM et le disque dans leurs retranchements les plus extrêmes.

2.4 Difficultés rencontrées

La principale difficulté de ce début de projet n'a pas été technique, mais **méthodologique**. Nous avons eu du mal à évaluer la "profondeur" de notre sujet :

— **Estimation difficulté :** Au départ, avions eu des idées de projet qui sont considérés comme pas assez concret ou pas assez personnelle (pas assez de code, trop proche d'un projet de recherche). Il a fallu de nombreuses discussions et recherches afin d'avoir pu construire quelque chose d'assez solide et qui nous permet de ramener notre touche personnelle.

Sur le plan technique, nous pensons que nous pourions rencontrer deux obstacles :

— **Complexité de Cypher 25 :** La syntaxe de l'opérateur `allReduce` est très récente. Réussir à lui faire transporter des données personnalisées (coût/profit) a demandé beaucoup d'essais-erreurs.

— **Calibrage des ressources :** Il est complexe de limiter la mémoire des bases de données de manière assez stricte pour observer un débordement sur le disque, sans pour autant faire planter l'intégralité de notre ordinateur.

3. Calendrier et Jalons

L'organisation temporelle du projet est structurée pour valider progressivement notre hypothèse de résistance système :

Étape 1 : Fondations (Terminé)

- **Étude comparative des travaux de M2 :** Pour identifier les scénarios de rupture non explorés.
- **Théorie :** Mise en place de l'aspect théorique du graphe piégé et définition des règles de génération des coûts.

Étape 2 : Développement et Ingestion (Mars)

- **Générateur :** Codage complet du script Python pour produire des fichiers de données massives (plusieurs millions d'arêtes).
- **Scripts d'ingestion :** Automatisation du chargement des données vers PostgreSQL et Neo4j pour garantir des tests reproductibles.
- **Écriture des requêtes :** Construction du code Cypher 25 (`allReduce`) et de la requête SQL récursive.

Étape 3 : Campagne de Tests et Mesures (Avril)

- **Tests de montée en charge :** Exécution des benchmarks sur des graphes de tailles croissantes jusqu'au crash système.
- **Monitoring bas niveau :** Capture des statistiques d'utilisation du disque (fichiers temporaires SQL) et surveillance du comportement du Garbage Collector de la JVM (Neo4j).
- **Collecte des logs :** Extraction des plans d'exécution pour comprendre où la mémoire sature.

Étape 4 : Analyse et Restitution (Mai)

- **Synthèse statistique :** Comparaison des courbes de performance (Temps vs Mémoire) pour les deux moteurs.
- **Rédaction finale :** Mise en évidence des avantages et inconvénients de chaque approche (Agilité vs Solidité).
- **Interprétation :** Conclusion sur la viabilité de Cypher 25 pour des problèmes NP-Difficiles en environnement contraint.